

Factory

Game Design Document

Nathan Tebbs · v0.5.0 · 2026-07-12

Drop ore, route it through upgraders, cash it in. Build recirculating belt loops on a grid you pay to expand. The puzzle is squeezing the most value out of the least space.

Contents

1. Overview	4
1.1. Concept Summary	4
1.2. Target Experience	4
1.3. Influences & Reference Games	4
1.4. Scope & Platform	5
2. Core Design Pillars	5
3. Gameplay	5
3.1. The Core Loop	5
3.1.1. Micro Loop (seconds)	6
3.1.2. Mid Loop (minutes)	6
3.1.3. Macro Loop (hours / sessions)	6
3.2. Tension & Conflict	6
3.3. Failure States	6
3.4. Player Agency	6
4. Systems	7
4.1. The Value Model	7
4.2. Objects	7
4.2.1. Object Rules	8
4.3. Upgrade Axes	8
4.4. The Grid & Space	8
4.4.1. Progression Through Space	9
4.5. Economy & Balance	9
5. UI & UX	9
5.1. Visual Language	9
5.2. HUD & Overlays	10
5.3. Feedback & Juice	10
5.4. Controls & Input	10
6. Art Direction	11
6.1. Style & Constraints	11
6.2. Color System	11
6.3. Object Visual Design	11
6.4. Animation & Motion	12
7. Audio	12
7.1. Music	12
7.2. Sound Effects	12
8. Technical Notes	12
8.1. Simulation Model	12
8.2. Save & Load	13
8.3. Mod / Data-Driven Support	14
8.4. Assets & Distribution	14
9. Roadmap	14

9.1. Milestone 1: Core Loop (done)	14
9.2. Milestone 2: Value Model & Loops (done)	14
9.3. Milestone 3: Economy & Space (done)	14
9.4. Milestone 4: Scale & Polish	15
10. Open Questions	15

1. Overview

1.1. Concept Summary

Factory is a value-loop belt builder in isometric 2D. A dropper emits ore. Belts carry it through upgraders that multiply its value. A furnace consumes the ore and banks that value as money. You spend the money on more grid and better equipment, and the loop tightens.

Ore is infinite: there is nothing to mine out and no deposit to relocate to. The whole game is **layout**: given a small grid and a stream of cheap ore, build the arrangement of belts and upgraders that turns it into the most money. The signature move is the recirculating loop, running one piece of ore past the same upgraders over and over to pump its value up before you let it reach the furnace.

The grid starts tiny and costs money to enlarge. So money pulls in two directions at once (spend it on space, or spend it on better machines), and space is always the thing you wish you had more of.

1.2. Target Experience

The feel we're after: a calm, constrained spatial puzzle that keeps paying out. Early on you can barely fit a straight line from dropper to furnace, and the ore you cash in is nearly worthless. As money comes in you buy a bigger grid and better parts, and suddenly a loop is possible, and a loop is worth far more than a line. The interesting decisions are all about **fitting**: how tight a loop can you pack, how many upgraders can you thread it through, when do you stop looping and cash out.

There is no win condition and no clock. A player who builds one gorgeous, dense loop is playing the game as intended just as much as one who tiles a huge grid with them. The satisfaction is watching value climb and knowing you earned it with the layout.

1.3. Influences & Reference Games

- **Miner's Haven**: The drop → route → upgrade → collect loop, recirculating ore through upgraders, and numbers that climb into the absurd. We take the loop and the object vocabulary (dropper, upgrader, furnace) directly, and drop the tycoon/rebirth grind around it.
- **Cities: Skylines**: The feel of a constrained isometric grid you expand deliberately, where space is the resource you actually manage.
- **Factorio**: Belts as one-item-per-cell transport lines, the joy of routing and packing. We borrow the transport model, not the extraction/tech-tree game.
- **Wookash (Anton Mikhailov)**: Code structure and entity design: fat structs, direct arrays, minimal abstraction.

1.4. Scope & Platform

- **Platform:** PC (macOS, Linux, Windows).
- **Engine/Stack:** mach.h, a single-header 2D C engine that grew out of this project and now lives in its own repo (github.com/vetr0s/mach.h, zlib licensed). It embeds RGFW for windowing and input, the engine's own OpenGL 3.3 core batch renderer, Clay for the UI, and stb_image for images. The game vendors a copy of the header at src/mach.h, so this repo still builds with nothing but a C compiler.
- **Solo:** Solo developer.
- **Art style constraint:** Isometric 2D, tile-based grid. The sprite pipeline is in (art in assets/sprites/ is baked into the binary at build time), but no sprites are drawn yet: every machine is still a procedural shaded block, and the ore is a procedural diamond that falls back automatically if no ore.png exists. Procedural rendering stays for the ground and for value-driven ore tinting (see Art Direction). Every object fits the same isometric footprint so layouts compose cleanly.

2. Core Design Pillars

1. **Complexity from simplicity.** Each object does one small thing. All the depth is in how you connect them on the grid. Interesting problems come from packing simple parts into tight space, not from special-case rules.
2. **The player is always in control.** No randomness, no hidden AI. Every ore's path and value is a consequence of the layout you built. A bad payout is a layout you can see and fix.
3. **Progress is visible, not gated.** No tech tree. Every object and upgrade is buyable from the start; cost is the only gate. You grow by earning money and choosing what to spend it on.
4. **Scale is graceful.** Ten loops should play as smoothly as a hundred. The game gets richer as it grows, not more unmanageable. Zoom and clean rendering keep a big factory readable.

3. Gameplay

3.1. The Core Loop

Place a dropper, some belts, an upgrader or two, and a furnace. Watch ore ride the belts, gain value at each upgrader, and turn into money at the furnace. Spend the money on grid or gear. Repeat, tighter.

3.1.1. Micro Loop (seconds)

Place an object on a tile; see it appear and start working immediately. Rotate a belt to redirect the flow. Drop an upgrader into the path and watch the ore that crosses it jump in value. Every action has instant, legible feedback.

3.1.2. Mid Loop (minutes)

You're solving a layout problem: "I have room for one more upgrader, where does it buy the most?" or "can I bend this line into a loop without running out of cells?" You reroute belts, close a loop, decide how many laps the ore should take before it hits the furnace. The payout responds immediately, so you tune by watching.

3.1.3. Macro Loop (hours / sessions)

You bank enough to expand the grid or buy a better tier of dropper, upgrader, or belt, which opens layouts that weren't possible before. Early sessions are single lines and first loops. Later sessions are dense grids of tuned loops, and the question shifts from "can I make a loop" to "what's the most value-dense loop that fits."

3.2. Tension & Conflict

Space versus spend. Money buys two competing things: more grid, or better machines. Grid gives you room to build bigger loops; better machines make a given loop worth more. You never have enough of either, so every payout is a small decision about which way to lean.

Loop versus cash out. Ore gains less value on each additional pass through your upgraders (see the value model). Looping longer earns more but ties up belt and space that another ore could be using. Knowing when a loop has stopped paying and the ore should go to the furnace is the core skill.

You express that decision through **layout**, not through a dial. A splitter sheds every other ore that reaches it, so where you tap it into the ring, and how many you tap in, is what sets the average number of laps an ore takes. One splitter on a long ring means many laps and a slow drain; two splitters, or a splitter on a short ring, means fewer laps and a fast one. Tuning the drain rate against the value curve is the loop-building puzzle.

3.3. Failure States

None. It's a sandbox; you can't lose. "Doing poorly" means a layout that earns less than it could: a loop that's too loose, an upgrader wasted on ore that's already near its ceiling. Nothing breaks. You notice because the number climbs slower than you know it could, and you go fix the layout.

3.4. Player Agency

- **Placement.** Where each object goes shapes everything. Dense cluster or spread out? The grid is small, so every cell spent is a cell you don't have.

- **Routing and loops.** A straight line, a single loop, nested loops, shared corridors feeding one furnace, all valid, all different in value and in how much space they eat.
- **Spend priority.** Grid, droppers, upgraders, belts. Which one unlocks your next jump depends on what your current layout is starved for.
- **Loop length.** How many laps before the furnace. Sets the trade between value squeezed out and throughput / space tied up.
- **Scaling style.** Perfect one small dense loop, or tile the grid with many. Both reach big numbers; the difference is taste.

4. Systems

4.1. The Value Model

This is the heart of the game. An ore carries a value. Passing through an upgrader multiplies that value. A furnace banks it as money. Two rules make the loop interesting:

Diminishing per pass. Each ore has a value **ceiling** it climbs toward as it passes upgraders. Every pass adds less than the one before: a log-shaped climb. Early passes multiply the value a lot; once the ore nears its ceiling, another pass barely moves it. So a single loop **settles**: running ore around forever asymptotes and stops paying. This is the soft anti-runaway rule. It replaces an earlier hard rule (an upgrader could only touch a given ore once), which had the side effect of making loops pointless, exactly the thing we now want to be central.

The ceiling is uncapped. The ceiling itself is not a fixed number. Ore quality (set by the dropper) and upgrader quality **combine and multiply** into a higher ceiling: better ore plus better upgraders make ore that scales far higher before it settles. Because the ceiling rises without limit as you invest, the game's numbers are **meant** to run away into the quadrillions and beyond. That's a feature, not a problem to tame. The soft cap bounds a single loop; it never bounds the game.

Upgraders drive both sides of this. A stronger upgrader climbs toward the ceiling faster **and** lifts the ceiling higher, and it still gets more out of cheap, low-ceiling ore than a weak upgrader would.

4.2. Objects

Five object types. Depth comes from many **variants** of each (tiers), not from more types. Each occupies one isometric tile.

- **Dropper.** Emits ore onto the tile it faces, on a fixed cadence. The dropper sets the ore's base quality, which feeds the ceiling. Ore is free and infinite.

- **Belt.** Carries one ore per cell in its facing direction. One item per cell; a packed line backs up and jams, a line with a gap advances. Belts are how you route and how you build loops.
- **Upgrader.** Its own object, but physically a belt surface: it moves ore **and** multiplies its value as the ore crosses it. Routing ore through (and around, and back through) upgraders is the whole layout puzzle.
- **Splitter.** A belt surface with **two** outputs, alternating between them: the first ore goes straight on, the next peels off down the branch, and so on. This is the only object that can take ore **out** of a loop, and a loop without one is a trap: every other object has a single facing, so ore that enters a cycle rides it forever and eventually packs the cycle into a permanent jam. A loop is a ring of belts and upgraders with a splitter tapped into it, shedding every other ore toward the furnace while the rest go around again.
- **Furnace.** Consumes ore and banks its accumulated value as money. The end of every path.

4.2.1. Object Rules

- **Grid-based placement.** One object per tile.
- **Directional.** Droppers, belts, upgraders, and splitters have a facing, set on placement and rotatable. A splitter carries a second facing for its branch, turned independently. Furnaces just consume whatever arrives.
- **One item per cell.** Belts, upgraders, and splitters hold one ore at a time; this is what makes packing and loop timing matter.
- **Splitters never wedge.** A splitter prefers the output its alternation points at, but takes the other one if that output is backed up. Strict alternation would deadlock a loop exactly when it filled, which is when draining it matters most.
- **Synchronized sim.** Everything ticks at one fixed rate. No per-object scheduling.

4.3. Upgrade Axes

Money buys better tiers along two axes today:

1. **Droppers:** better ore, a higher ceiling to climb toward.
2. **Upgraders:** stronger multiplier, faster climb per pass and a higher ceiling.

A third axis, **belts** (faster and tighter loops, more passes per second), is designed but not built: belt speed is still the one global sim tick, so a per-belt cadence has to land before a belt can have a tier at all. A fourth, **furnace tiers**, is deferred. One furnace type is enough for now (see Open Questions).

4.4. The Grid & Space

The world is an isometric grid. The **playable** region starts tiny and is enlarged with money.

Expansion cadence. The unlocked square grows on a power-of-two side length: it starts at 4×4 (the smallest square that fits the starter dropper, belt and furnace) and doubles to 8×8 , 16×16 , and so on, up to a 128×128 cap inside the 256×256 world grid. Each step roughly quadruples the cells you have. Cost scales with the area unlocked, so expansion is a real decision, not a formality.

Space is the constraint. At 4×4 you can't fit a loop worth running: it's dropper, belt, furnace, cashing in near-worthless ore in a straight line. Room to loop is something you **buy**. That's the pressure the whole economy hangs on: every expansion is the difference between a line and a loop, or between a loop and a denser one.

4.4.1. Progression Through Space

- **Early:** One small region. Straight lines only. Tiny payouts that bootstrap the first expansion.
- **Mid:** Enough grid and belt to close a loop. A loop beats a line, so payouts jump. You start trading grid against gear.
- **Late:** Room for many loops, or a few very dense ones. The puzzle is value-density: the most value squeezed from the fewest cells. Numbers climb hard.

4.5. Economy & Balance

Two sinks, one currency. Money is spent on grid expansion and on gear tiers (droppers, upgraders, belts). They compete: space enables bigger loops, gear makes any loop worth more. There is no dominant order; what to buy next depends on what your layout is starved for.

Numbers explode by design. Because the ceiling is uncapped and better ore and upgraders multiply into it, values are meant to grow into the quadrillions and past. We do not temper this. The diminishing-per-pass curve only limits a **single loop**; the game's totals are unbounded and that's the point.

Legibility early. Base ore is worth little and moves one cell at a time, so early play reads clearly: you can watch a single ore climb. Scale hides the individual pieces later, which is fine; by then you think in loops, not ores.

5. UI & UX

5.1. Visual Language

Isometric perspective. A 45-degree view over the grid. Machines and ore are drawn procedurally today, authored to the isometric footprint sprites will use (see Art Direction); the ground is procedural too. Objects are sorted back-to-front so the scene reads as depth without any 3D.

Color encodes type and state. Object color signals function (dropper, belt, splitter, upgrader, furnace each read distinctly). Ore value can read through color or brightness as it climbs. Outline brightness signals state: active, idle, or blocked/jammed.

One footprint. Every object fits the same tile, so a dense factory stays scannable and layouts compose without alignment fuss.

5.2. HUD & Overlays

- **Top-left:** Money. Simple text.
- **Top-right:** The shop: the object/tier to place, selected one highlighted, every price on screen at all times (a row you cannot afford is greyed out rather than hiding its cost), plus the grid-expansion button.
- **Top-center:** Inspect: what sits under the cursor, machine and/or ore.
- **Bottom-center:** The controls bar.
- **When placing:** A ghost preview of the object on the hovered tile. Green if placeable; hard red if the cell is taken or still locked; faint red if the cell is fine but you cannot afford the piece, so a click that buys nothing is never a surprise.
- **Expansion:** The cost of the next grid step, and a clear boundary between unlocked and locked cells.

5.3. Feedback & Juice

- **Object placed:** A short pop and click. Satisfying, not distracting.
- **Ore carried:** Ore slides smoothly cell to cell along belts; you can follow one piece around a loop.
- **Value gained:** A small tick or flash when an upgrader bumps an ore's value.
- **Cashed out:** A clear beat when the furnace banks an ore and money jumps.
- **Grid expanded:** New territory fades in, a satisfying reveal of room to build.

5.4. Controls & Input

Mouse-driven. Click to place. Deleting is a tool you select (like any other), not a second click gesture. Scroll to zoom. Pan with arrow keys / WASD. Escape opens the pause menu.

Keyboard shortcuts (current bindings):

- 1–6: pick dropper / belt / upgrader / furnace / delete / splitter. Pressing the active tool again clears it.
- R: rotate the piece under the cursor in place; over an empty tile, rotate the facing of the next piece placed.
- T: turn a splitter's branch (which quarter-turn its second output takes).
- E: buy the next grid expansion. - / =: step the tier bought down / up.
- Space: pause / unpause the simulation. Freezes the world; you can still build and pan while paused.
- Escape: pause menu (resume / main menu). On the title screen, Escape quits.
- Backtick: toggle the debug overlay.

- K / L: save / load. Temporary, until the menu's Continue is the only entry point.

6. Art Direction

6.1. Style & Constraints

Sprites for the machines and ore. The plan is for real sprites to replace the shaded blocks, for the five machine types (dropper, belt, splitter, upgrader, furnace) and for the ore. The pipeline is built and the hook is live: a PNG dropped into `assets/sprites/` is baked into the binary at build time, and the ore already asks for its sprite and falls back to the procedural diamond when there is none. What is missing is the art itself: `assets/sprites/` is empty, so everything still draws as a block today. Sprite work is authored in Aseprite. Everything draws through the engine's `mach_r2d_*` 2D batch renderer.

Procedural rendering stays where it earns its place. The ground checker is drawn in code: it is viewport-culled and scales with the grid, so a sprite would be pure overhead. Value-driven ore tinting also stays procedural: the “hotter” color as an ore climbs is data-driven from its value, not something to hand-paint. Sprites cover the parts that read as objects; code covers the parts that are really data.

One tile footprint. Sprites are authored to the engine's isometric footprint: a 64×32 pixel 2:1 diamond (`MACH_ISO_TILE_W = 64`, `MACH_ISO_TILE_H = 32`), with 28 pixels of vertical body per unit of block elevation (`MACH_ISO_ELEV = 28`). Every object fits the same tile, so composition stays grid-clean and there is no asset sprawl.

Authored against the engine palette. Sprites use the engine's existing modus-vivendi palette (`MACH_COLOR_*`), not ad-hoc colors, so the new art and the remaining procedural rendering stay coherent while the transition is half done.

Clarity first. A factory should be readable at a glance before it's pretty. Art is **guided** by how the game plays, not the other way around.

6.2. Color System

- **Object function:** Each of the five types gets a distinct base color so a dense grid is scannable.
- **Ore value:** Read through color or brightness: higher-value ore looks visibly “hotter.”
- **State:** Outline brightness for active / idle / blocked. Jammed belts should be obvious.

This grammar lets you understand a big factory without clicking into anything.

6.3. Object Visual Design

- **Dropper:** Distinct silhouette that reads as a source. The tallest piece on the grid.
- **Belt:** Thinner surface with an animated flow direction (scrolling chevrons in its facing).

- **Splitter:** A belt surface with two exits, drawn as two arrows on its top face: the one the next ore will take is lit, the other dim.
- **Upgrader:** A belt surface marked to read as “this one boosts.”
- **Furnace:** Visually the endpoint: a shallow open bin with a recessed mouth, clearly a sink.

6.4. Animation & Motion

- **Belts:** Surface chevrons scroll in the flow direction, running even when empty.
- **Ore:** Slides smoothly between cells; you can track one piece around a loop.
- **Value gain:** A brief flash/pop when an upgrader fires.
- **Grid reveal:** New territory fades in over about half a second.

Minimal, clear, satisfying. No motion for its own sake.

7. Audio

Secondary, but it carries a lot of the feel. Deferred until the loop is solid.

7.1. Music

One ambient, loopable track. Calm, rhythmic, lightly industrial. It sits under the sound design and sets mood; it should never compete. One track is enough to ship.

7.2. Sound Effects

Critical:

- Object placed: short percussive hit.
- Belt running: soft mechanical hum, looping and quiet, signals the factory is alive.
- Ore cashed out: a small chime as the furnace banks it.

Nice-to-have:

- Upgrader fires: a subtle tick as value jumps.
- Grid expanded: a satisfying reveal sound.

All subtle and loopable, nothing grating on repeat, mutable.

8. Technical Notes

8.1. Simulation Model

Fixed-timestep, synchronous sim. The world steps at a constant rate (currently 3/s), decoupled from the render framerate. Every dropper, belt, splitter, upgrader, and furnace ticks together. Deterministic and predictable.

Discrete grid + render interpolation. Belt movement is a discrete grid sim: one ore per cell, advanced a cell per tick. The renderer interpolates each ore from its previous cell to its current one so it slides smoothly. This is the Factorio transport-line model, deliberately **not** continuous physics; the discrete grid is what makes packing, jamming, and loop timing exact.

Entities are fat structs in flat arrays. No generic component system. Each object type is a full struct; the sim loops the arrays directly. Grid indices map cells to the object/ore on them. The whole **World** is one arena allocation.

Dead-end ore. Ore with nowhere to go does not jam forever. Ore that hits a dead end (off the grid edge, bare ground, or a dropper's back) tips off the belt and is gone. Deleting an entity despawns the ore sitting on its cell the same way.

The sim carries no visual timing. The tick that kills a dead-ended ore emits a `World_Event` (`WORLD_EVENT_FELL`, or `WORLD_EVENT_BANKED` at a furnace) recording only what happened, with no notion of how long anything should look like it takes. The renderer drains that queue once a frame and turns each event into a transient real-time **Effect** (`effects.c`), aged in seconds rather than ticks. So the drop-out and the payout pop animate on the render clock while the sim stays a clean 3/s integer machine, and neither can drift into the other. `world_despawn` and `item_kill` in `world.c`; `game_sync_effects` in `game.c`.

Big numbers. Values are `i64` today, which carries them a long way. Because the design wants numbers to run into the quadrillions and beyond, every running total (money, a furnace's tally, an ore's ceiling) saturates at a clamp rather than adding plainly: an `i64` that wraps negative would brick a run, and this design drives values at the roof on purpose. Storage and display will eventually need more care still (wider or arbitrary-precision integers, and human-readable large-number formatting).

One bit per upgrader, per ore. "Each distinct upgrader lifts the ceiling once" is enforced by giving every ore a bitmap of the upgraders that have already lifted it, so the cap on upgraders in the world is the width of that bitmap: 256 today, 32 bytes per ore. That is far past what the largest region (128×128) can hold usefully, but it is a real ceiling, and raising it costs 8 bytes per ore per 64 upgraders.

8.2. Save & Load

Unit of save: full world state: every object and its tier, the ore in flight, money, the unlocked grid size, camera position/zoom.

Format: a versioned binary blob, written field by field rather than as a raw struct dump, so it stays stable across the compilers on the three target platforms. Not human-editable; the magic and version are checked on load, and a file that fails either (or is truncated, or names an upgrader id that cannot exist) is refused rather than half-loaded. The format is at v3, and every version back to v1 still loads.

Persistence: manual save, one slot, no branching saves. Auto-save is designed but not built: today the only ways in and out are the menu's Continue and the temporary K / L keys.

8.3. Mod / Data-Driven Support

Initial: objects, tiers, and sim rules are hardcoded in C.

Future: data-driven object/tier definitions (speed, cost, multiplier, ceiling contribution) so tiers can be added and rebalanced without recompiling. Scripting (e.g. Lua) only if a concrete need appears.

8.4. Assets & Distribution

Assets are baked into the executable. Sprites and any other assets are embedded at build time, never loaded from a directory next to the binary. A single self-contained static binary per platform is a hard project constraint: there is nothing to install and nothing to place alongside the executable.

Prebuilt binaries. The game ships prebuilt static binaries for macOS (universal arm64 + x86_64), Linux (x86_64, glibc 2.35+), and Windows (x86_64). They are built and published automatically on each git tag.

9. Roadmap

9.1. Milestone 1: Core Loop (done)

Place dropper, belt, upgrader, furnace. Ore drops, rides belts, gains value at upgraders, banks at the furnace. Directional placement with rotation, one item per cell with correct jamming, smooth item interpolation, money HUD. This exists today.

9.2. Milestone 2: Value Model & Loops (done)

Replaced the hard once-per-upgrader rule with the diminishing-per-pass climb toward an uncapped, ore-plus-upgrader ceiling. Recirculating loops are now the strongest play. The exact curve constants are still open tuning (see Open Questions), but the model ships.

9.3. Milestone 3: Economy & Space (done)

Money sinks: grid expansion on the 2^n cadence, and buyable tiers for droppers and upgraders. This is what turned the sandbox into a game with a progression: space versus spend, line versus loop. Belt tiers are the one piece that did not land, because belt speed is still the global sim tick (see Upgrade Axes). The cost and tier curve ships untuned by feel and wants a balance pass.

Two things landed alongside it that the milestone did not name. The **splitter** (a belt surface with two outputs, alternating between them) closed a hole in the core design:

every object has one facing, so ore entering a loop rode it forever, which made the recirculating loop, this game's signature move, physically unbuildable. The splitter is the tap that drains a loop. And **save/load** arrived early, pulled forward from Milestone 4.

9.4. Milestone 4: Scale & Polish

Sound, balance passes, and real art. Ship-ready.

Already done ahead of this milestone: viewport-culled rendering (the ground, the machines and the ore are each culled to the screen), save/load, and the sprite pipeline, which landed early to unblock art work in Aseprite (see Art Direction). What is left here is the art itself, the audio, and tuning.

10. Open Questions

Decisions to make as development continues:

- **How many furnaces?** One furnace total (everything must route to a single sink, a strong spatial constraint), or many placeable furnaces? Undecided, and it meaningfully changes the layout puzzle.
- **Furnace tiers.** Whether and when the furnace becomes a fourth upgrade axis (better cash-out), or stays a single fixed object.
- **The exact curves.** The precise diminishing-per-pass shape, and exactly how dropper (ore) quality and upgrader quality combine into the ceiling. This is tuning, settled by playtest.
- **Grid-expansion cost curve.** How steeply each 2^n step should cost, so expansion always feels earned but never stalls.
- **Big-number representation.** When i64 stops being enough, and what to move to.